



ioBroker.pondpump

Handbuch — OASE AquaMax Eco Titanium Teichpumpen in
ioBroker einrichten, steuern und überwachen



Einsteigerfreundliche Anleitung

1. Was der Adapter macht



Wie der Adapter die Pumpen erreicht

Wie ioBroker deine Pumpen erreicht: heute über die OASE-Cloud und — im lokalen Modus — direkt über dein LAN.

Der **pondpump**-Adapter verbindet ioBroker mit deiner/deinen **OASE AquaMax Eco Titanium** Teichpumpe(n) über das Gateway **OASE Garden Controller Cloud (EGC)**. Sobald er läuft, kannst du aus ioBroker heraus (und damit aus VIS, Skripten, Szenen, Alexa usw.):

- **jede Pumpe ein- und ausschalten**,
- die **Pumpendrehzahl** von 0 – 100 % einstellen,
- **Live-Telemetrie** lesen: Leistung (W), Motordrehzahl (U/min), Temperatur (°C) und Netzspannung (V),
- den **Verbindungs- und Gerätestatus** sehen.

Jede Pumpe behält den Namen, den du ihr in der OASE-App gegeben hast (z. B. *Wasserfall*, *Filter*), damit du sie im Objektbaum wiedererkennt.



Gut zu wissen: Der Adapter spricht mit dem **Controller** (Artikel 55317), der wiederum mit den Pumpen (Artikel 73656) kommuniziert. Er ist ein eigenständiges Projekt, unabhängig vom Community-Adapter für die OASE-Steckdosen-Controller — er zielt auf die smarten Teichpumpen.

2. Bevor du startest — was du brauchst

Du brauchst	Wofür
Eine laufende ioBroker -Installation (js-controller, Node.js ≥ 22)	Die Plattform, auf der der Adapter läuft
Einen OASE Garden Controller Cloud (EGC, Artikel 55317), in der OASE-App eingerichtet	Das Gateway, mit dem sich der Adapter verbindet
Eine oder zwei OASE AquaMax Eco Titanium Pumpen (Artikel 73656), in der App gekoppelt	Die gesteuerten Geräte
Deine Pumpen laufen bereits in der OASE-App	Der Adapter nutzt denselben Cloud-Account
Ein Cloud-Refresh-Token (siehe Kapitel 4)	Damit sich der Adapter ohne dein Passwort anmeldet



Tipp: Bring zuerst alles in der **OASE-App** zum Laufen. Wenn die App die Pumpen schalten kann, kann es der Adapter auch.

3. Adapter installieren

Der Adapter ist auf npm als **iobroker.pondpump** veröffentlicht. Solange er noch nicht Teil des offiziellen ioBroker-Repositories ist, installierst du ihn aus der Quelle:

1. Öffne die ioBroker-**Admin**-Oberfläche.
2. Gehe zu **Adapter** und schalte den **Experten-Modus** ein (Zauberhut-Symbol oben rechts).
3. Klicke auf das Symbol „**Aus eigener URL installieren**“ (Katze/Octocat).
4. Gib eines davon ein:
 - den npm-Paketnamen **iobroker.pondpump** oder
 - die GitHub-URL eines Release-Tarballs, falls du eine erhalten hast.
5. Bestätige und warte, bis der Adapter in der Liste erscheint.
6. Klicke auf das **+** auf der pondpump-Kachel, um eine **Instanz** anzulegen (**pondpump.0**).

Die Instanz-Konfiguration öffnet sich automatisch. Lass sie kurz offen — zuerst brauchen wir einen Refresh-Token (nächstes Kapitel).

4. Cloud-Refresh-Token beschaffen (Schritt für Schritt mit mitmproxy)

Die OASE-Cloud nutzt **Microsoft Azure AD B2C** zur Anmeldung. Aus Sicherheitsgründen speichert der Adapter dein Kontopasswort **nicht**. Stattdessen verwendet er einen **Refresh-Token** — eine lange, einmalige Anmeldeinformation, die deine OASE-App beim Login erhält. Diesen Token fängst du **einmal** mit einem kleinen Werkzeug namens **mitmproxy** ab, trägst ihn in den Adapter ein, und danach erneuert der Adapter ihn automatisch.

Keine Sorge, falls du so etwas noch nie gemacht hast — folge einfach den Schritten genau.



mitmproxy sitzt zwischen Smartphone und OASE-Cloud

mitmproxy sitzt zwischen deinem Smartphone und der OASE-Cloud, sodass du den Login mitlesen und den Token kopieren kannst.

4.1 mitmproxy installieren und starten

mitmproxy ist ein kleines, kostenloses Programm. Wir nutzen seine Browser-Variante **mitmweb**. Folge den Schritten für **dein** Betriebssystem.

Windows (mit PowerShell)

1. Öffne deinen Webbrowser und gehe auf <https://mitmproxy.org/downloads/>.
2. Lade das **Windows**-Paket herunter (die neueste Version — meist ein `.msi` -Installer).
3. Öffne die heruntergeladene Datei und klicke dich durch den Installer: **Weiter** → **Weiter** → **Installieren** → **Fertig**.
4. Öffne nun **PowerShell**:
 - Drücke die **Windows-Taste**, tippe `PowerShell` und klicke in der Liste auf **Windows PowerShell**.
 - Es erscheint ein dunkles Fenster mit blinkendem Cursor — das ist die Kommandozeile.
5. Tippe diesen Befehl ein und drücke **Enter** (er legt den Proxy-Port eindeutig auf **8080**):

```
mitmweb --listen-port 8080
```

6. Fragt Windows nach dem **Netzwerkzugriff**, klicke **Zulassen**. Es öffnet sich automatisch die Oberfläche unter <http://127.0.0.1:8081> im Browser deines Rechners — das ist das mitmproxy-Bedienfeld, das du beobachtest. Der eigentliche **Proxy lauscht auf Port 8080** und wartet dort auf den Smartphone-Datenverkehr. ✓
7. **Lass dieses PowerShell-Fenster die ganze Zeit offen** — schließt du es, stoppt mitmproxy. Zum Beenden später ins Fenster klicken und **Strg + C** drücken.



„mitmweb wird nicht erkannt“? Schließe PowerShell und öffne es erneut (damit es das neu installierte Programm bemerkt). Hast du stattdessen die `.zip` -Variante geladen, entpacke sie, tippe dann in PowerShell `cd` gefolgt vom Ordnerpfad und starte `.\mitmweb.exe`.

macOS

1. Öffne das **Terminal** (**Cmd + Leertaste**, `Terminal` tippen, Enter).
2. Am einfachsten mit **Homebrew**: `brew install mitmproxy`. (Kein Homebrew? Lade die macOS-Version von <https://mitmproxy.org/downloads/> und entpacke sie.)
3. Starte `mitmweb --listen-port 8080`. Ein Browser-Tab öffnet sich unter <http://127.0.0.1:8081>.

Linux

1. Installiere es mit `pipx install mitmproxy` (oder über das Paket deiner Distribution bzw. die Binaries von der Download-Seite).
2. Starte `mitmweb --listen-port 8080` in einem Terminal und öffne <http://127.0.0.1:8081>.

In jedem Fall gilt: Die Browser-Seite auf **:8081** ist das Bedienfeld, das du beobachtest, und über **Port 8080** schickt gleich dein Smartphone seinen Datenverkehr (nächster Schritt).

4.2 Den Smartphone-Verkehr über mitmproxy leiten

Smartphone und Rechner müssen im **selben WLAN** sein.

1. Ermittle die **lokale IP deines Rechners** (z. B. `192.168.1.20`): Windows `ipconfig`, macOS/Linux `ip addr / ifconfig`.
2. Am Smartphone: **WLAN-Einstellungen** → **dein Netzwerk** → **Proxy** → **Manuell** und trage ein: **Server = IP deines Rechners**, **Port = 8080**. Speichern.
3. Öffne den Browser am Smartphone und rufe <http://mitm.it> auf. Wähle dein Handy-System, **installiere** das angebotene Zertifikat **und vertraue ihm**:
 - **iOS**: Profil installieren, dann *Einstellungen* → *Allgemein* → *Info* → *Zertifikatsvertrauen* und das mitmproxy-Zertifikat **einschalten**.
 - **Android**: als **CA-Zertifikat** installieren (Einstellungen → Sicherheit → Verschlüsselung & Anmeldedaten → Zertifikat installieren → CA-Zertifikat).

Dieses Zertifikat ist der Schlüssel, mit dem mitmproxy den sonst verschlüsselten OASE-Verkehr lesen kann. **Entferne den Proxy und das Zertifikat danach wieder.**

4.3 Login mitschneiden und den Refresh-Token holen

1. Leere in der **mitmweb**-Seite die Liste (damit neue Anfragen leicht zu finden sind).
2. In der **OASE-App**: **abmelden**, dann **wieder anmelden**.
3. Tippe im **Filterfeld** oben in mitmweb einen dieser Filter ein — das ist der Trick, mit dem du dir das Scrollen durch Hunderte Einträge sparst:

Diesen Filter eingeben	Zeigt
<code>~u token</code>	nur Anfragen, deren URL „token“ enthält
<code>~d account.oase.com</code>	nur Anfragen an den OASE-Login-Server
<code>~b refresh_token</code>	nur Anfragen, deren Body <code>refresh_token</code> enthält

4. Klicke die **POST**-Anfrage an, die auf `/oauth2/v2.0/token` endet.
5. Öffne den Reiter **Request** und sieh dir den Formular-Body an. Suche `refresh_token=` und kopiere den langen Wert dahinter (bis zum nächsten `&`).
 - **Zusatz-Tipp**: Drücke in mitmweb `/` und suche nach `refresh_token`, um es sofort hervorzuheben.
6. Trage diesen Wert in die Adapter-Einstellung „**Cloud-Refresh-Token**“ ein (Kapitel 5).



Der Refresh-Token ist lang (mehrere hundert Zeichen) — kopiere ihn **vollständig**. Behandle ihn wie ein Passwort: nicht weitergeben. Du kannst ihn jederzeit widerrufen, indem du dich in der OASE-App überall abmeldest. **Dein Kontopasswort wird nie in den Adapter eingegeben.**

4.4 (Fortgeschritten) Gerätepasswort für den lokalen Modus finden

Nur nötig, wenn du den Verbindungsmodus **local** möchtest (Kapitel 8). Während mitmproxy noch läuft:

1. Öffne in der App deinen Teich, damit die Pumpen geladen werden (das löst den Inventar-Download aus).
2. Tippe im Filterfeld von mitmweb **~u Inventory** ein, um die Anfrage an **/User/Inventory** anzuzeigen.
3. Klicke sie an und öffne den Reiter **Response**. Suche im JSON die **Custom-Attribute** der Pumpe; der Eintrag mit **Id = 101** enthält das **Gerätepasswort** — ein **64-stelliger** Wert (er kann **\uXXXX**-Escape-Sequenzen enthalten, das ist in Ordnung).
4. Kopiere diesen Wert in die Adapter-Einstellung „**Gerätepasswort**“. Der Adapter dekodiert ihn und nutzt ihn für den lokalen TLS-Handshake.



Falls mitm.it nicht lädt: Prüfe, ob der Proxy des Smartphones auf die IP deines Rechners und Port 8080 zeigt und ob Datenverkehr fließt. Bei iOS musst du das Zertifikat sowohl **installieren** als auch **ihm vertrauen** (zwei getrennte Schritte).

5. Instanz konfigurieren

Öffne **Instanzen** → **pondpump.0** → **Einstellungen** (Schraubenschlüssel-Symbol). Die Einstellungen sind gruppiert:

Verbindung

Einstellung	Was du einträgst
Verbindungsmodus	cloud für den Internet-Weg, local für den hausinternen LAN-Weg (Kapitel 8). Die beiden schließen sich gegenseitig aus.
Abfrageintervall	Wie oft (Sekunden) der Adapter den Status liest. Standard 30 . Minimum 5.

Cloud

Einstellung	Was du einträgst
Cloud-Refresh-Token	Der Token aus Kapitel 4 (verschlüsselt gespeichert).
<i>Erweitert (Base-URL, Token-URL, Client-ID, Scope)</i>	Standardwerte belassen, außer OASE ändert die Cloud.

Lokal (nur für `local`)

Einstellung	Was du einträgst
Controller-IP	Die Adresse des EGC-Controllers in deinem Netzwerk.
Gerätepasswort	Das 64-stellige Gerätepasswort (fortgeschritten; siehe Kapitel 8).
Bind-Adresse / Port	Adresse des ioBroker-Hosts und TCP-Port, zu dem sich der Controller zurückverbindet (Standard 5999).

Klicke auf **Speichern**. Die Instanz startet und nach wenigen Sekunden sollte `info.connection` auf `true` springen.

6. Die Objekte, die der Adapter anlegt

Nach der ersten erfolgreichen Abfrage findest du unter `pondpump.0` :

pondpump.0	
└─ info.connection	(true bei Verbindung)
└─ <gateway>	der EGC-Controller (device)
└─ serialNumber, firmware	
└─ online	(Controller erreichbar)
└─ pumps.<deviceNumber>	ein device je Pumpe, benannt wie in der App
└─ control.on	← ein/aus (schreibbar)
└─ control.speed	← Drehzahl 0–100 % (schreibbar)
└─ control.speedRaw	← Drehzahl 0–255 (roh) (schreibbar)
└─ status.connected	Pumpe erreichbar
└─ status.fcStatus	Statustext des Controllers
└─ telemetry	
└─ power	Live-Leistung in W
└─ speed	Live-Motordrehzahl in U/min
└─ temperature	°C
└─ temperature2	°C (zweiter Sensor)
└─ voltage	Netzspannung in V
└─ raw.sensorN	noch nicht zugeordnete Sensorwerte

7. Pumpen steuern und auslesen

Pumpe ein-/ausschalten — setze `pumps.<deviceNumber>.control.on` auf `true` / `false`.

Drehzahl einstellen — schreibe einen Prozentwert (0–100) nach `pumps.`

`<deviceNumber>.control.speed`. Der Adapter sendet den Befehl, bestätigt ihn und liest die Pumpe kurz darauf erneut, sodass die Zustände die Realität widerspiegeln.

Telemetrie lesen — die Werte unter `telemetry` aktualisieren sich live bei jeder Abfrage (schnelle Werte wie Leistung und Drehzahl in jedem Zyklus, langsamere wie Temperatur nur alle paar Zyklen, um die Cloud zu schonen). Nutze sie in VIS, in Diagrammen oder in Skripten.

Beispiel (JavaScript-Adapter):

```
// Pumpe "Wasserfall" auf 70 % laufen lassen
setState('pondpump.0.pumps.1234567.control.speed', 70);

// Ihre Live-Leistung protokollieren
on('pondpump.0.pumps.1234567.telemetry.power', (obj) => {
  log('Pumpenleistung: ' + obj.state.val + ' W');
});
```


8. Lokaler Modus (hausinternes LAN)

Der Adapter kann **komplett über dein lokales Netzwerk** laufen, ohne Internet. Stelle den **Verbindungsmodus** auf **local**, dann:

- startet ioBroker einen kleinen **TLS-Server** und sendet ein **UDP-Weck-Paket** an den Controller,
- der Controller **verbindet sich zurück** über TLS und authentifiziert mit dem **Gerätepasswort**,
- danach liest der Adapter Gateway + Pumpen, pollt **Live-Telemetrie** (Leistung, Drehzahl, Temperatur, Spannung) und lässt dich **ein-/ausschalten und die Drehzahl setzen** — alles über LAN.

Was du brauchst:

- **Controller-IP** — die Adresse des EGC-Controllers in deinem Netzwerk.
- **Gerätepasswort** — der 64-stellige Wert (wie du ihn ausliest, steht in Kapitel 4.4).
- einen offenen Netzwerkpfad: **UDP 5959** hinaus zum Controller und **TCP 5999** zurück zu ioBroker. Liegen Controller und ioBroker in unterschiedlichen Subnetzen/VLANs, gib diese zwei Richtungen frei.

Lass die **Bind-Adresse** auf **0.0.0.0** — der Adapter ermittelt automatisch die Host-Adresse, zu der der Controller sich zurückverbinden soll.



Hinweis: Der aktuelle **Drehzahl-Sollwert** (der %-Wert) wird über den lokalen Kanal nicht zurückgelesen — der State `control.speed` zeigt den zuletzt in ioBroker gesetzten Wert. **An/Aus wird live gelesen** (aus der Leistungsaufnahme der Pumpe) und spiegelt damit auch Änderungen aus der OASE-App.

9. vis-2-Widgets

Der Adapter bringt **drei fertige vis-2-Widgets** mit — du musst nichts extra installieren. Sobald der Adapter installiert ist, wird vis-2 automatisch neu gestartet, und die Widgets stehen im vis-2-Editor unter der Widget-Gruppe „**Pond Pump**“ bereit.



Tipp: Erscheinen die Widgets nach der Installation nicht sofort im Editor, lade die Editor-Seite im Browser einmal neu (**Strg + F5**).

9.1 Widget einfügen und mit einer Pumpe verbinden

1. Öffne den **vis-2-Editor** und ziehe eines der beiden Widgets aus der Gruppe „**Pond Pump**“ auf deine Ansicht.
2. Wähle rechts in den Widget-Einstellungen unter **Instanz** deine `pondpump` -Instanz (z. B. `pondpump.0`).

3. Wähle darunter unter **Pumpe** die gewünschte Pumpe aus — die Liste zeigt automatisch alle erkannten Pumpen mit ihrem Namen.

Mehr musst du nicht einstellen: Die Widgets kennen die passenden Objekt-IDs selbst und verbinden sich automatisch mit der gewählten Pumpe.

9.2 Widget „Pumpen-Visualisierung" (PumpVisual)

Dieses Widget stellt die Pumpe grafisch dar:

- Ein **Flügelrad** dreht sich abhängig von der Drehzahl — in 10-%-Stufen, von langsamem Kriechen bis schnell.
- Ist die Pumpe **aus**, steht das Flügelrad still und ein **rotes Kreuz** liegt darüber.
- Ist die **Saisonale Durchflussregelung (SFC)** aktiv, dreht sich statt des Flügelrads ein **Eiskristall**.
- Darunter stehen die Live-Werte: **Leistung** (W), **Drehzahl** (rpm) und **Power** (Sollwert in %).
- Liegt eine **Wassertemperatur** vor (State `telemetry.waterTemperature`), rückt das Flügelrad nach links und rechts erscheint ein **gefülltes Thermometer** samt Wert; die Füllung ist je nach Temperatur eingefärbt (kalt = blau bis warm = bernstein). Ohne Wert bleibt die Darstellung wie bisher.

Oben rechts zeigt ein farbiges Abzeichen den Zustand: **Läuft**, **Aus** oder **Saison-Modus**.

9.3 Widget „Pumpen-Steuerung" (PumpControl)

Mit diesem Widget steuerst du die Pumpe:

- **Ein / Aus** schaltet die Pumpe.
- Der **Schieberegler** stellt die Leistung (Power) in Prozent ein. Das Kommando wird erst beim Loslassen gesendet, damit das Gerät beim Ziehen nicht mit Befehlen überflutet wird. Die **Schnellwahl-Tasten** (0/25/50/75/100 %) setzen den Wert direkt.
- Die **Saisonale Durchflussregelung (SFC)** lässt sich mit dem Button aktivieren/deaktivieren. Der Button reagiert sofort (mit Spinner, bis das Gerät bestätigt) — mehrfaches Drücken ist nicht nötig.



Was ist SFC? „Seasonal Flow Control" ist OASEs temperaturabhängige, saisonale Durchflussregelung: Bei aktivem SFC senkt die Pumpe Fördermenge und Förderhöhe automatisch (bis zu -50 %) und passt sich so übers Jahr an die Teichbiologie an. Es ist **kein Frostschutz**.

9.4 Widget „Scheduler-Status" (PumpScheduler)

Dieses Widget zeigt auf einen Blick, **was der eingebaute Zeitplan-/Regel-Scheduler** (Kapitel 10–11) gerade mit der Pumpe macht — und **warum**:

- Ein **Status-Abzeichen**: **Scheduler aktiv**, **Manuell / aus** (keine gültige Planung) oder **Notlauf** (Sensorausfall).
- Die aktuelle **Ausgabe** in % groß dargestellt — daneben ein **kleines Flügelrad**, das sich (optional animiert) mit der realen Drehzahl dreht — dazu der Betriebszustand (Läuft/Aus/Saison-Modus), **Tag/Nacht** und die **Zielleistung** des Schedulers.
- **Begründungs-Chips**: woraus die Basis kommt (**Temperaturkurve**, **Zeitfenster** oder **Grundlast**) und welche Modifikatoren gerade greifen (**Nachtschutz**, **Wetter-Boost**, **Frost-Halt**).
- Das **aktive Fenster**, der Zeitpunkt der **nächsten Änderung** sowie **Sonnenauf-/untergang**.
- **Aktoren**: Ist im Zeitplan ein **Aktor-Fenster** konfiguriert (Wasserfall, Bachlauf, Sauerstoffpumpe ...), wird jeder Aktor **über der Telemetrie** zeilenweise als **Icon – Name – Flügelrad** angezeigt. Name und Icon vergibst du im Zeitplan-Editor (Modus „Aktor“); ohne Namen steht „Aktor 1“, „Aktor 2“ ... Das kleine **hellgrüne Flügelrad** dreht sich (optional animiert), solange der Aktor **an** ist, und steht still (gedimmt), wenn er aus ist. In den Widget-Einstellungen (Bereich „Aktoren“) lässt sich **jeder Aktor einzeln** ein- oder ausblenden.
- **Wassertemperatur** (farbig), **Leistung** (W) und **Drehzahl** (rpm).
- Eine **Steuerleiste** mit den Grundfunktionen (An/Aus, Leistungs-Schnellwahl, SFC). Ein Hinweis erinnert daran, dass der Scheduler manuelle Eingriffe beim nächsten Durchlauf wieder überschreiben kann.

Die Werte kommen aus neuen, schreibgeschützten States `pumps.<Nr>.schedule.*`, die der Scheduler bei jeder Auswertung pflegt — sie lassen sich auch in eigenen Skripten oder der History nutzen.

9.5 Aussehen anpassen

In den Widget-Einstellungen unter **Darstellung** kannst du u. a. die **Akzentfarbe** wählen, den **Kartenhintergrund** ausblenden, die **Animation** abschalten oder einzelne Bereiche (Werte, Ein/Aus-Tasten, Schnellwahl, SFC, Steuerleiste, Telemetrie) ein- und ausblenden.

10. Zeitpläne (Pumpen nach Zeitplan betreiben)

Der Adapter kann jede Pumpe nach einem **Tagesplan** betreiben statt mit einer festen Einstellung. Du definierst Zeitfenster, die je eine **Power %** setzen oder **SFC** ein-/ausschalten; außerhalb jedes Fensters fällt die Pumpe auf eine einstellbare **Grund-Power** zurück.

10.1 Zeitplanbetrieb für eine Pumpe einschalten

1. Öffne die Instanz-Einstellungen und bleibe auf dem Tab **Verbindung**.
2. Scrolle ganz nach unten zum Abschnitt **Zeitpläne**. Dort sind alle vom Adapter erkannten Pumpen aufgelistet. (Ist die Liste leer, lass den Adapter einmal laufen, damit er die Pumpen findet, und lade die Seite neu.)
3. Schalte die Pumpe(n) ein, die nach Zeitplan laufen sollen. Jede aktivierte Pumpe erhält oben einen eigenen Tab „**Scheduler – <Pumpenname>**“.

10.2 Zeitfenster definieren

Öffne den **Scheduler**-Tab der Pumpe:

- **Grund-Power %** — gilt immer dann, wenn kein Fenster aktiv ist (z. B. ein ruhiges Nacht-Niveau).
- Die Tabelle enthält die **Zeitfenster**. Füge mit **Zeitplan hinzufügen** eine Zeile hinzu und setze:
 - **Start / Ende** — Tageszeiten (HH:MM). Ein Fenster darf nicht über Mitternacht gehen — teile es in zwei auf.
 - **Modus** — **Power %** (das Fenster setzt eine feste Leistung), **SFC** (das Fenster schaltet die Seasonal Flow Control ein oder aus) oder **Aktor** (das Fenster schaltet einen **externen State**, z. B. Bachlauf/Wasserfall/UVC — kombinierbar mit Astro-Grenzen, siehe 10.4).
 - **Wert** — der Leistungsprozentwert bzw. an/aus für SFC; beim **Aktor** die **Ziel-Objekt-ID** plus An-Wert (aktiv) und optionalem Aus-Wert (inaktiv; leer = außerhalb unverändert lassen).
- Fenster **dürfen sich nicht überschneiden**. Der Editor prüft live und zeigt eine rote Meldung, wenn sich zwei Fenster überlappen; der Adapter prüft zusätzlich vor dem Anwenden — ein ungültiger Zeitplan wird nie ausgeführt.

Nicht vergessen zu **speichern**.

10.3 Wie der Zeitplan ausgeführt wird

Der Adapter wertet den Zeitplan aus und wendet das Ziel **an jeder Fenstergrenze** an (und einmal beim Start):

- In einem **Power**-Fenster setzt er diese Leistung und schaltet SFC aus.
- In einem **SFC**-Fenster schaltet er SFC in den gewählten Zustand und behält die Grund-Power (die nur zählt, solange SFC aus ist).
- Außerhalb jedes Fensters wendet er die **Grund-Power** mit SFC aus an.

Er schreibt nur, wenn sich das Ziel tatsächlich ändert — so verträgt sich der Zeitplan mit der manuellen Steuerung: Deine letzte manuelle Änderung bleibt, bis die nächste Fenstergrenze die Pumpe erneut setzt.

10.4 Astronomische Zeitfenster (Sonnenauf-/untergang)

Start und Ende eines Fensters müssen keine feste Uhrzeit sein. Bei jeder Grenze kannst du statt **Uhrzeit** auch **Sonnenaufgang** oder **Sonnenuntergang** wählen und einen **Versatz in Minuten** angeben (auch negativ). Beispiele: Start = „Sonnenuntergang + 0“, Ende = „Sonnenaufgang + 120“ ergibt ein **Nachtfenster**, das über Mitternacht läuft; „Sonnenaufgang – 30“ beginnt eine halbe Stunde vor Sonnenaufgang. Die Sonnenzeiten werden täglich neu berechnet.



Hinweis (Forschung): Eine **Nacht-Absenkung des Durchflusses ist im Sommer kontraproduktiv** (das Sauerstoff-Minimum liegt nachts). Astro-Fenster eignen sich vor allem als **Schutzfenster** (nicht reduzieren) und für Nebenaktoren (Bachlauf/Wasserfall). Siehe [doc/research/](#) .

Standort: Für die Sonnenzeiten braucht der Adapter einen Standort. Auf dem Tab **Verbindung** im Abschnitt **Zeitpläne** wählst du den **Standort-Modus**:

- **ioBroker-Systemstandort verwenden** (Standard) — nimmt Breite/Länge aus den ioBroker-Systemeinstellungen.
- **Ein Standort für alle Pumpen** — eine gemeinsame Position; setze sie auf der **Karte** (klicken oder Marker ziehen), per **Adress-Suche** oder in den Feldern Breite/Länge.
- **Ein Standort pro Pumpe** — jede Pumpe legt ihre Position auf ihrem eigenen Tab fest.

Nachtschutz: Im Abschnitt **Temperatur-/Wettersteuerung** kannst du je Pumpe den **Nachtschutz** aktivieren. Dann wird der Durchfluss **während der astronomischen Nacht nicht unter einen Boden** (Standard 100 %) gesenkt, sofern die Wassertemperatur $\geq$ einer Schwelle (Standard 18 °C) liegt — genau das, was die Forschung empfiehlt (das Sauerstoff-Minimum liegt nachts; eine Absenkung im Warmen ist schädlich). Der Nachtschutz braucht einen Standort und wird durch die **Maximalleistung** weiterhin gedeckelt.

11. Temperatur- und wetterabhängige Steuerung

Zusätzlich zu festen Zeitfenstern kann jede Pumpe **nach Wassertemperatur und Wetter** geregelt werden. Die Grundidee aus der Teich-Forschung (siehe [doc/research/](#)): Der **Durchfluss richtet sich nach der Wassertemperatur** (kälteres Wasser = weniger Umwälzung, wärmeres = mehr), und **Wetter darf den Durchfluss nur anheben**, nie senken (z. B. Hitze → mehr Belüftung).



Wichtig: Der Pumpen-State `telemetry.temperature` ist die **Geräte-**, nicht die Wassertemperatur. Nutze als Kurvenquelle einen **echten Wassersensor** (z. B. einen Teichfühler eines anderen Adapters).

Öffne den **Scheduler**-Tab der Pumpe und scrolle zum Abschnitt **Temperatur-/Wettersteuerung**.

11.1 Zusammenspiel mit den Zeitfenstern

Das Auswahlfeld oben legt fest, wie die Kurve zu den Zeitfenstern steht:

- **Kurve überschreibt das aktive Zeitfenster** — die Wassertemperatur-Kurve bestimmt immer die Leistung.

- **Kurve gilt nur außerhalb der Zeitfenster** — innerhalb eines Fensters gewinnt das Fenster, außerhalb greift die Kurve.

11.2 Wassertemperatur-Sensor und Kurve

Ganz oben wählst du den **Wassertemperatur-Sensor**: Das Dropdown listet die geräteeigenen Temperatursensoren der Pumpe **mit ihrem aktuellen Wert** — vergleiche sie mit einem Thermometer und wähle den, der die Wassertemperatur zeigt. Die Auswahl speist den neuen State `telemetry.waterTemperature` und **belegt die Kurvenquelle vor**. Einen **externen** Fühler wählst du stattdessen auf „keiner“ und trägst sein Objekt unten als Kurvenquelle ein (Lupen-Symbol). In beiden Fällen spiegelt `telemetry.waterTemperature` die **tatsächlich genutzte Kurvenquelle** — zeigt also auch den Wert deines externen Sensors, nicht nur einen geräteeigenen.

So aktivierst du die Kurve:

1. Schalte **Wassertemperatur** → **Leistungskurve** ein.
2. Prüfe die **Wassertemperatur-Quelle** — durch den Sensor oben vorbelegt; für einen externen Fühler sein Objekt über das **Lupen-Symbol** wählen.
3. Trage **Stützpunkte** ein (Temperatur °C → Power %). Zwischen den Punkten wird linear interpoliert, unter-/oberhalb geklemmt. Mit **Standardkurve laden** setzt du die empfohlene Kurve aus der Forschung (Q10-Regel, 17 °C → 100 %) als Startpunkt.

Sicherheitsverhalten: Fällt der Sensor aus (Quelle liefert keinen Wert), läuft die Pumpe zur Sicherheit auf **100 %** — zu viel Umwälzung kostet nur Strom, zu wenig kostet Fische.

11.3 Feinabstimmung (optional)

Unter der Kurve stehen optionale Grenzwerte. Jedes Feld zeigt in der Admin-Oberfläche einen **Wertvorschlag** (grauer Platzhalter) und einen **Hilfetext**:

- **Mindestleistung %** — Untergrenze, der Durchfluss fällt nie darunter. Empfohlen **35–40 %**.
- **Maximalleistung %** — Obergrenze, die Leistung übersteigt diesen Wert nie (auch nicht bei Boost oder Notlauf). Z. B. **90 %** für eine Pumpe, die nur bis dahin läuft.
- **Temperatur-Glättung (Stunden)** — mittelt die Temperatur, damit kurze Ausschläge die Pumpe nicht ständig verstellen. Empfohlen **12–24**, **0** = aus.
- **Hysterese (°C)** — die Kurve wird erst nach einer Temperaturänderung um diesen Betrag neu berechnet. Empfohlen **0,5–1**, **0** = aus.
- **Max. Änderung (% pro Stunde)** — begrenzt, wie schnell sich die Leistung ändert (sanftes Anfahren). Empfohlen **10–20**, **0** = sofort.

11.4 Wetterregeln

Die Regeltabelle darunter kann den Durchfluss zusätzlich **anheben** oder Aktoren schalten. Jede Regel vergleicht eine **Quelle** (beliebige State-ID, z. B. Außentemperatur oder ein Regen-OID) über einen Operator

(`<` , `≤` , `>` , `≥` , `=` , `≠`) mit einem **Schwellwert**. Trifft sie zu, greift ihr **Effekt**:

- **Auf Leistung % anheben** — hebt die Leistung auf mindestens diesen Wert (senkt nie).
- **Auf 100 % anheben** — volle Leistung (z. B. Hitze).
- **Halten (Frost)** — friert die Leistung auf dem letzten Wert ein.
- **SFC ein/aus** — schaltet die Seasonal Flow Control der Pumpe.
- **Aktor setzen** — schreibt eine beliebige **Ziel-Objekt-ID** auf einen Wert (true/false oder Zahl), z. B. einen Belüfter oder Bachlauf einschalten.

Alle zutreffenden Regeln werden kombiniert (Anhebungen nehmen das Maximum; ein „Halten“ friert ein, sofern keine Anhebung gewinnt; Aktor-Schreibvorgänge summieren sich). Der Adapter abonniert die Quell-States und wertet **sofort bei Änderung** neu aus, nicht nur an Fenstergrenzen.



Hinweis zu SFC: Regelt die Kurve die Leistung, während die **native SFC** der Pumpe eingeschaltet ist, überschreibt die Pumpe den Sollwert — der Adapter warnt dann im Log. Schalte SFC aus oder steuere sie über eine „SFC“-Regel.

Umstieg von 0.3.0: Das Regelmodell hat sich geändert. Regeln aus 0.3.0 (Effekte *Power %/SFC/Aus*) sind inaktiv und müssen mit den neuen Effekten neu angelegt werden.

Nicht vergessen zu **speichern**.

12. Fehlerbehebung

Symptom	Was zu prüfen ist
<code>info.connection</code> bleibt false	Ist ein Refresh-Token eingetragen? Fange einen neuen ab (Kapitel 4) — Tokens können ablaufen, wenn du dich woanders anmeldest.
Log meldet AUTH FAILED	Der Refresh-Token ist ungültig/abgelaufen → einen neuen abfangen.
Keine Pumpen erscheinen	Sind die Pumpen in der OASE-App online? Der Adapter spiegelt das Cloud-Inventar.
Befehle bewirken nichts	Warte auf die erste erfolgreiche Abfrage (dann lernt der Adapter die Pumpen-Adressierung). Prüfe das Log.
Mehr Details gewünscht	Setze das Log-Level der Instanz auf debug — jeder Schritt wird mit einem Tag wie <code>[poll]</code> , <code>[cloud/auth]</code> , <code>[cloud/cmd]</code> , <code>[schedule]</code> , <code>[astro]</code> , <code>[geocode]</code> protokolliert. Geheimnisse werden nie geloggt.
Eine Pumpe läuft mit unerwarteter Leistung	Im <code>debug</code> -Level protokolliert jeder Scheduler-Durchlauf die komplette Entscheidungskette der Pumpe — siehe unten.

Die Log-Zeilen sind nach Komponente getaggt, sodass sich jedes Problem eindeutig eingrenzen lässt. Füge bei einer Fehlermeldung das Debug-Log rund um den Fehler bei.

Das Scheduler-Entscheidungslog lesen

Im `debug` -Level gibt jede Scheduler-Auswertung je Pumpe exakt aus, warum sie die Leistung/SFC so gewählt hat:

- eine **tick**-Zeile mit der aktuellen Uhrzeit und allen Roh-Quellwerten,
- eine **inputs**-Zeile: rohe / geglättete / gemappte Wassertemperatur (mit Glättung τ und Hysterese K), der aufgelöste Sonnenauf-/untergang und Tag/Nacht sowie Priorität / Min / Max-Leistung,
- eine **decision**-Zeile: die Basis (aus Kurve oder aktivem Fenster), die $Q_{\min}$ -Untergrenze, der Nachtschutz, jede greifende Wetterregel, die Aktor-Fenster und der $Q_{\max}$ -Deckel bis zur finalen Leistung/SFC,
- der **Rampen-/Halte**-Status und der **nächste Auswertungs**-Zeitpunkt.

So verrät eine einzige `[schedule] pump 1 decision: ...` -Zeile die komplette Begründung — kein Rätselraten mehr, warum eine Pumpe auf einem bestimmten Prozentwert steht.

13. Datenschutz & Sicherheit

- Dein **OASE-Kontopasswort** wird nie in den Adapter eingegeben oder gespeichert.
- Der **Refresh-Token** und das **Gerätepasswort** werden in ioBroker **verschlüsselt** gespeichert.
- Der Adapter spricht nur mit der OASE-Cloud (oder, im lokalen Modus, direkt mit deinem Controller).
- Nutzung auf eigenes Risiko — dies ist ein inoffizielles Community-Projekt, nicht mit der OASE GmbH verbunden.

Fragen oder Probleme? Öffne ein Issue im [GitHub-Repository](#) des Projekts. Viel Freude am Teich! 🐟